Lab 3: Information Retrieval

Session 3. RAG: A practical introduction with LangChain.

Goal

This lab session aims to introduce the new paradigm of text generation combined with information retrieval, specifically Retrieval-Augmented Generation (RAG) architectures, gaining hands-on experience with a simple approach to document summarization. The main objective is to demystify the complex landscape of advanced natural language processing by developing a practical RAG system that can retrieve and synthesize information from news collections. Through this immersive learning experience, students will explore the innovative LangChain framework, a powerful tool that simplifies the development of sophisticated language processing applications. By designing a RAG architecture capable of generating summaries in two distinct linguistic modes —one tailored for young readers with simplified language and another crafted for domain experts using advanced terminology —students will develop a nuanced understanding of how artificial intelligence can adapt information presentation to different audience needs. This lab session introduces students to the technical intricacies of semantic search strategies, language generation and provide a practical framework for understanding how modern language technologies can transform information retrieval and comprehension in diverse contexts.

Bibliography

1. Rothman D. (2024). RAG-driven Generative AI, Build custom retrieval augmented generation pipelines with LlamaIndex, Deep Lake, and Pinecone. Denis Rothman. First Edition. Packt Publisher.
2. Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., ... & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive nlp tasks. Advances in Neural Information Processing Systems, 33, 9459-9474.
3. Wu, S., Xiong, Y., Cui, Y., Wu, H., Chen, C., Yuan, Y., Huang, L., Liu, X., Kuo, T., Guan, N., & Xue, C.J. (2024). Retrieval-Augmented Generation for Natural Language Processing: A Survey. ArXiv, abs/2407.13193. Information Processing Systems 33 (2020): 9459-9474.
4. Oshin M & Campos N. (2014) Learning LangChain. O'Reilly Media.
5. Auffarth B. (2023). Generative AI with LangChain: Build large language model (LLM) apps with Python, ChatGPT and other LLMs. Packt Publisher.
6. https://python.langchain.com/docs/introduction/

What is RAG?

Retrieval-Augmented Generation (RAG) is an advanced natural language processing technique that enhances large language models by dynamically incorporating external knowledge into the response generation process. RAG bridges the gap between the fixed knowledge embedded in large language models and the need for up-to-date, domain-specific, or contextually precise information. Instead of relying solely on the model's pre-trained parameters, RAG allows the model to:

1. Retrieve relevant information from an external knowledge base.
2. Use this retrieved context to generate more informed and accurate responses.

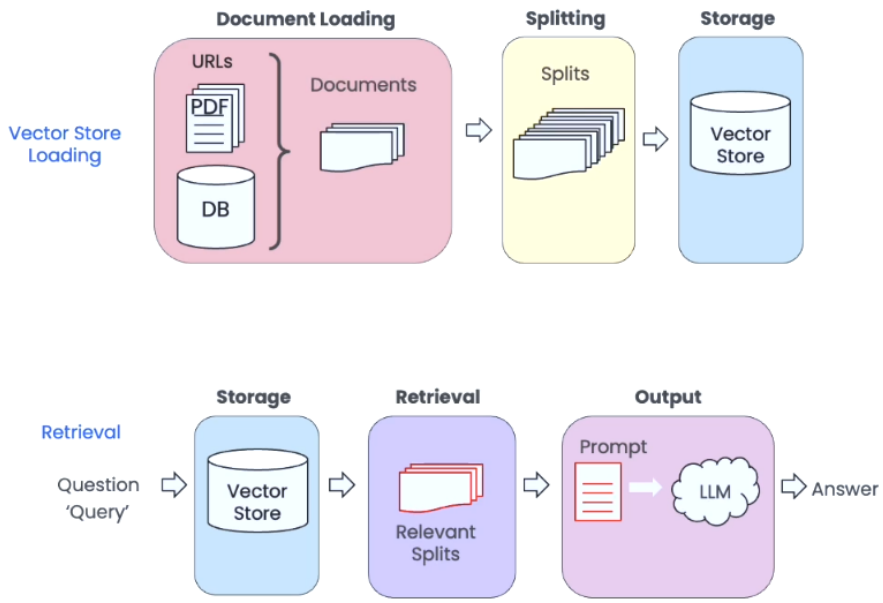

Figure 1. A simple Retrieval Augmented Generation Architecture

As can be seen in Figure 1, like classical information retrieval the RAG architecture has two main phases, the first one dedicated to indexing the information, and the second one dedicated to retrieving information. Conversely to the classical information retrieval models, the RAG architecture uses a LLM model guided by a prompt to obtain the answer given an input query.

Key Components in RAG

Document Loading

Document loading is the initial phase where raw data from various sources is ingested into the RAG system. This process involves:

- Using document loaders to handle diverse data formats such as PDF, TXT, CSV, DOCX files, emails, and many more including images and videos.
- Extracting, parsing, cleaning, and formatting text from these sources to prepare them for further processing.
- Converting different data types into a standardized format, typically plain text.

Splitting

After loading, documents are split into smaller, manageable segments. This phase is crucial for several reasons:

- It helps in handling large documents that exceed token limits of language models (e.g., GPT-3's 2048 token limit)

- It improves the efficiency of retrieval and processing in later stages

Splitting strategies include:

- Fixed-size chunking: dividing text into segments of a predetermined length.
- Content-aware chunking: splitting based on the structure or content of the document.

The choice of splitting strategy depends on the document structure and specific requirements of the RAG system.

Storage

The storage step involves indexing and storing the processed document chunks for efficient retrieval. Key aspects of this step include:

- Converting documents chunks into vector embeddings using models like BERT, SentenceTransformer or GPT.
- Storing these embeddings in a vector database or other suitable data structure.
- Creating indexes to facilitate quick and relevant retrieval of information.

Storage systems often include:

- Vector databases optimized for similarity searches.
- Metadata storage for additional context and filtering capabilities.

This phase ensures that the processed and split documents are readily available for the retrieval stage of the RAG pipeline, allowing for efficient and accurate information retrieval based on user queries.

Question Query

The RAG process begins when a user submits a query. This query can range from a simple question to a complex problem requiring detailed context. The query is typically sent to an orchestrator, which manages the flow of information throughout the RAG workflow.

Retrieval

The retrieval step is crucial for finding relevant information to augment the language model's knowledge. This phase involves:

- Search for information: Retrieval algorithms scan through knowledge bases, databases, or external sources to find relevant information related to the user's query.
- Data vectorization: The retrieved information is transformed into vector embeddings, which are numerical representations in high-dimensional space. This facilitates classification and analysis.
- Relevance ranking: The retrieval model ranks the obtained information according to its relevance to the query. The most pertinent documents or fragments are selected for the next step.

Prompt

The prompt phase involves crafting an effective input for the language model:

- Augmentation: The orchestrator enriches the original query with the retrieved context, amplifying the input given to the language model.
- Forming the augmented query: This step creates an enhanced version of the original query, supplemented with additional, retrieved information.
- Prompt template: A prompt template is used, which includes placeholders for all the information to be passed to the LLM. This typically includes a system prompt, historical context, and augmented query.

LLM to Generate the Output

In this last step, the LLM model processes the augmented prompt to generate the response:

- Context processing: The LLM interprets the augmented prompt, which includes the original query, retrieved information, and any additional context or instructions.
- Generation: Drawing from both the augmented prompt and its internal knowledge, the LLM synthesizes an engaging answer tailored to the user's query.
- Response formulation: The model generates a coherent, natural language response that effectively addresses the user's inquiry, grounded in the relevant facts and knowledge provided by the retrieval phase.

By combining these phases, the RAG architecture enables the generation of more accurate, up-to-date, and contextually relevant responses compared to using an alone LLM.

Several frameworks and APIs have been proposed for designing and implementing RAG systems. These frameworks simplify the complex process of building AI applications that combine document retrieval with generative AI. LangChain is one of the most popular and comprehensive frameworks for building RAG systems. It provides a flexible, modular approach to creating intelligent AI applications.

In this lab session, we leverage LangChain's components to build a simple RAG system. By utilizing LangChain's retrieval and language model interfaces, we will develop a practical workflow that seamlessly connects document retrieval mechanisms with generative summarization techniques.

LangChain Framework

LangChain[1] represents a transformative development in the AI ecosystem, serving as a sophisticated framework that bridges the gap between large language models and practical language applications. At its core, LangChain provides developers with a comprehensive toolkit for building applications that can reason, remember, and interact with external data sources, all while maintaining a clean and modular architecture.

[1] https://python.langchain.com/docs/introduction/

A particular aspect of LangChain is its abstraction layer for language models. This layer provides a unified interface to interact with various LLM providers, including industry leaders like OpenAI and Anthropic, as well as open-source alternatives. This abstraction enables developers to switch between different language models without significant code changes, providing flexibility and reducing the time of implementation. The framework supports both cloud-based and local model deployments, accommodating different deployment scenarios and security requirements.

Installing LangChain and dependencies

```
#Installing LangChain
!pip install langchain
!pip install langchain-community
#Installing additional dependencies
!pip install openai
!pip install sentence-transformers
!pip install faiss-cpu
!pip install pypdf
!pip install pydub
!pip install --upgrade httpx openai
!pip install tiktoken
!pip install chromadb
!pip install --upgrade --quiet lark langchain-chroma
```

Documents

The Document class in LangChain Python is a fundamental component for managing and manipulating text data within the framework. It provides a structured representation of documents, enabling various operations such as retrieval, transformation, and storage. The Document class encapsulates content, metadata, and additional information related to the document. It seamlessly

integrates with other LangChain components, allowing for efficient workflows and data processing pipelines. The class can handle various document formats, including plain text and JSON, providing flexibility in data handling.

A Document object typically consists of two main fields:

1. *page_content*: A string containing the raw text of the document.
2. *metadata*: A dictionary storing additional information about the text, such as source URL, author, or other relevant data.

To create a Document object, you can use the following example.

Example 1: Creating and updating a Document

```
from langchain.schema import Document

doc = Document(
   page_content="This is the content of the document.",
   metadata={"author": "John Doe", "date": "2023-10-01"}
)
print(doc.page_content)  # Outputs: This is the content of the document.
print(doc.metadata)      # Outputs: {'author': 'John Doe', 'date': '2023-10-
01'}

doc.page_content = "Updated content for the document."
doc.metadata["updated"] = "2023-10-02"
```

Data Loader

Loaders are essential components that enable the integration of diverse data sources into AI applications. These loaders create documents from various sources, allowing users to load data as documents from configured sources. Loaders can handle a wide range of data sources, including APIs, databases, and file systems.

They optimize data retrieval and processing for faster response times. Moreover, loaders allow developers to tailor loaders to specific needs. Some of the common loaders are:

1. TextLoader: Loads textual data from various sources.
2. CSVLoader: Handles CSV file loading.
3. UnstructuredFileLoader: Processes unstructured file formats.
4. UnstructuredHTMLLoader: Parses and loads HTML content.
5. JSONLoader: Handles JSON data loading.
6. PyPDFLoader: Specializes in loading PDF documents.
7. ArxivLoader: Retrieves academic papers from arXiv.
8. Docx2txtLoader: Converts and loads Microsoft Word documents.
9. DirectoryLoader: Loads documents from a directory, supporting wildcard patterns.

Loaders play a crucial role in preparing data for further use within LangChain applications. The loaders return a list of *Documents* objects.

Example 2: Loading a CSV file.

```
#Loading data from a csv file
from langchain.document_loaders import CSVLoader
loader = CSVLoader(file_path='./MN-DS-news-classification.csv',
                   csv_args={"delimiter": ",",
                             "quotechar": '"',
                             "fieldnames":
["data_id","id","date","source","title","content","author","url","published"
,"published_utc","collection_utc",   "category_level_1","category_level_2"]
                             },
                   content_columns=["title", "content"],

metadata_columns=["data_id","id","date","source","author","url","published",
"published_utc","collection_utc",    "category_level_1","category_level_2"],
```

```
                    encoding="utf-8",


                    )
newsdata = loader.load()
print(len(newsdata))
```

It is important to note that the CSVLoader does not automatically recognize headers in CSV files. To properly use the CSVLoader, you must first remove the header row from the CSV file.

Text Splitters

TextSplitter in LangChain is an interface for splitting text into smaller, manageable chunks. It is a crucial component for processing large documents or datasets in natural language processing tasks. TextSplitter allows setting the maximum size of text chunks. Also, the splitters enable specifying overlap between chunks to maintain context. They support custom functions for measuring chunk length and options to strip whitespace from chunk ends. The next example illustrates how to use the Splitter to segment documents.

Example 3: Splitting Documents

```
text_splitter = TokenTextSplitter(
       chunk_size = 256,
       chunk_overlap = 32,
       add_start_index = True)
splits = text_splitter.split_documents(documents)
```

The TokenTextSplitter is particularly useful when it is needed to process large texts that might exceed a LLM context window. This class splits text into smaller chunks, based on tokens, while

preserving semantic meaning. Ensures chunks stay within token limits for LLM processing. The main parameters are:

- *chunk_size:* Maximum number of tokens per chunk.
- *chunk_overlap*: Number of tokens that overlap between chunks.
- *add_start_index:* Determines whether to add metadata about the starting index of each chunk.
- *encoding_name:* The tokenizer encoding to use (default is *"cl100k_base"* for GPT models).

VectorDatabases

Vector databases in LangChain are specialized data stores designed to efficiently store, index, and retrieve high-dimensional vector representations of data, particularly text embeddings. These databases enable semantic similarity searches, which are crucial for various natural language processing tasks and retrieval-augmented generation applications**.** LangChain provides a standard interface for working with vector stores, allowing easy switching between different implementations. Two of the most popular vector databases in LangChain are Chroma and FAISS.

Chroma

Chroma is an open-source embedding database that simplifies building LLM applications. It offers:

1. Rich querying and filtering capabilities.
2. Seamless integration with LangChain (Python and JavaScript).

3. Scalability from local development to production clusters.

FAISS

FAISS (Facebook AI Similarity Search) is a library developed by Facebook AI Research for efficient similarity search and clustering of dense vectors, it offers:

1. High performance for large-scale datasets.
2. Various indexing methods for different use cases.
3. Integration with LangChain for vector storage and retrieval.

VectorDatabases in LangChain require an embedding method to be explicitly specified. The embedding method determines how the text will be converted into numerical vectors. Without specifying an embedding method, the vector database will not know how to create the vector representations needed for similarity search.

LangChain provides interfaces to integrate embeddings from various sources and models. Through its embedding interfaces, it is possible to use embeddings from OpenAI, HuggingFace, Cohere, and other providers. This flexibility allows you to choose the embedding solution that best fits your needs.

Example 4: Using Embedding from LangChain

```
!pip install sentence_transformers

import openai
from langchain_community.embeddings import (
   OpenAIEmbeddings,
   HuggingFaceEmbeddings
)

# Using OpenAI embeddings
```

```
openai_embed = OpenAIEmbeddings(api_key  = 'XXXXXXXX')
# Using HuggingFace embeddings
hf_embed = HuggingFaceEmbeddings(model_name="sentence-transformers/all-
mpnet-base-v2")
```

Example 5: Indexing documents with Chroma

```
#Example using OpenAI embedding
client = chromadb.Client() # Create a Chroma client object.
vectordb = Chroma(client=client, embedding_function=openai_embed, persist_directory='databasePath',
collection_name="collectionName")
vectordb.add_documents(documents) # Add documents to the Chroma collection.

#Example using HuggingFace embedding
client = chromadb.Client() # Create a Chroma client object.
vectordb = Chroma(client=client, embedding_function=hf_embed, persist_directory='databasePath',
collection_name="collectionName")
vectordb.add_documents(documents) # Add documents to the Chroma collection.
```

Following document indexing, the second phase of the RAG architecture commences. This involves retrieving relevant information from indexed sources to augment the available context and subsequently generate the output answer.

Retrieving Information

Retrieval strategies in LangChain are a crucial component for efficiently finding and extracting relevant information from large datasets. They are closely related to the VectorDatabase used to indexing the documents. They serve as an interface between unstructured queries and structured data, enabling semantic search and enhancing the capabilities of language models in various applications. LangChain offers several retrieval strategies to enhance efficiency and accuracy, three of the most common are:

1. Similarity Search (SS): The basic method that returns documents most similar to the query vector.
2. Maximal Marginal Relevance (MMR): A technique that balances relevance and diversity in retrieved documents.
3. Contextual Compression (CC): Extracts the most relevant segments from retrieved passages, reducing irrelevant information.

For SS and MMR searches, the retrieval strategy hinges on the indexing method employed within the Vector Database. In the case of CC retrieval, the strategy integrates a filtering mechanism (compression) to process the documents. This compression aims to generate concise versions of documents that maximize relevance to the query. Usually, the compression process uses an LLM to obtain the relevant information in the document.

Example 6: Retrieving documents using SS

```
query = "murder in shooting of black people"
ss = vectordb.similarity_search(query, k=10)
len(ss)
i=1
for x in ss:
   print(i, x.metadata['id'])
   i += 1
```

For similarity searches, both the query and the parameter *k* must be specified. *k* represents the desired number of documents to be retrieved.

Example 7: Retrieving documents using MMR

```
mmr = vectordb.max_marginal_relevance_search(query, k=10, fetch_k=50)
len(mmr)
i=1
for x in mmr:
   print(i, x.metadata['id'])
   i+=1
```

For maximal marginal relevance searches, the query, fetch_k, and the parameter *k* must be specified. *k* represents the desired number of documents to be retrieved. fetch_k controls the initial number of documents that are retrieved from the vector database before the MMR algorithm is applied. In this case, fetch_k=50 indicates that the system will initially fetch 50 documents that are most similar to the query based on simple similarity scores. These 50 documents then serve as the candidate pool for the MMR algorithm to select the final 10 most relevant and diverse documents.

Example 8: Retrieving documents using CC

```
from langchain.retrievers import ContextualCompressionRetriever
from langchain.retrievers.document_compressors import
LLMChainExtractor

def pretty_print_docs(docs):
   print(f"\n{'-' * 100}\n".join([f"Document {i+1}:\n\n------"
+d.metadata["id"]+"\n"+d.page_content for i, d in
enumerate(docs)]))

llm = OpenAI(temperature=0, model="gpt-3.5-turbo-instruct")
compressor = LLMChainExtractor.from_llm(llm)

compression_retriever = ContextualCompressionRetriever(
   base_compressor=compressor,
   base_retriever=vectordb.as_retriever(),
   max_documents=10
```

```
)

question = "murder in shooting of black people"
compressed_docs =
compression_retriever.get_relevant_documents(question)
pretty_print_docs(compressed_docs)
print(len(compressed_docs))
```

This example demonstrates how to use the *ContextualCompressionRetriever* in LangChain to efficiently retrieve and compress relevant documents. It leverages a combination of VectorDatabase search and LLM-based compression to provide concise and informative summaries of the most pertinent information.

Initially, the ContextualCompressionRetriever was imported from the langchain.retrievers module. This class is essential for retrieving and compressing documents based on their relevance to a given query. The ContextualCompressionRetriever requires a specified strategy to compress the retrieved documents. For this purpose, the LLMChainExtractor is imported from langchain.retrievers.document_compressors. This class relies on an LLM model that represents the core of the compression method. Therefore, the OpenAI LLM ("gpt-3.5-turbo-instruct") model is instantiated. Subsequently, the compression method is instantiated using the LLM model. Finally, the ContextualCompressionRetriever is instantiated, specifying the parameters:

- *compressor*: The compressor that will be used to summarize the retrieved documents.

- *base_retriever:* The underlying retriever used to fetch the initial set of relevant documents. This is assumed to be a vector database instance (like Chroma, FAISS, etc.) already populated with the documents.
- *max_documents:* Limits the number of documents that the retriever will attempt to retrieve and compress.

It is important to note that when instantiating the OpenAI LLM, the temperature parameter is set to 0. This ensures deterministic and more concise outputs. The temperature parameter controls the randomness of the LLM's output.
After addressing the challenges of indexing and retrieving documents within the RAG architecture, the next step involves crafting effective prompts and integrating the retrieved context from external sources into LLM chains to generate high-quality answers.

Prompts in LangChain

PromptTemplate in LangChain is a fundamental class for creating dynamic, reusable prompts for language models. It allows developers to define a template string with placeholders that can be filled with specific values at runtime, enabling the generation of customized prompts for various applications.

PromptTemplate uses placeholders (e.g., {variable_name}) within a string template to allow for dynamic content insertion. It validates the input variables against the defined template, ensuring all required variables are provided. PromptTemplate allows

specification of input types for variables, though all are assumed to be strings if not explicitly defined.

Example 9: PromptTemplate for answering questions

```
from langchain.prompts import PromptTemplate
template = """Use the following pieces of context to answer the
question at the end. If you do not know the answer, just say
that you do not know, do not try to make up an answer. Use
three sentences maximum. Keep the answer as concise as
possible. Always say "thanks for asking!" at the end of the
answer.
{context}
Question: {question}
Helpful Answer:"""
QA_CHAIN_PROMPT = PromptTemplate(input_variables= ["context",
"question"], template=template)
```

Example 10: PromptTemplate for summarizing documents.

```
template=""" Summarize this text:
               - Use simple, clear language
               - Maintain the core message
               - Use only four sentences

               TEXT: {context}

               SUMMARY:"""
SUMM_CHAIN_PROMPT= PromptTemplate(input_variables=["context"],
template=template)
```

Chains in LangChain

In LangChain, chains represent a sequence of calls, typically involving an LLM, but potentially also other components like tools

or data preprocessing steps. They allow you to orchestrate complex workflows by chaining together different components to achieve a desired output.

Key Concepts:

- *Composability*: Chains are designed to be easily combined and extended. You can create complex workflows by linking multiple chains together.
- *Flexibility*: LangChain offers a variety of chain types, each designed for specific use cases, allowing you to choose the most appropriate approach for your task.
- *Modularity*: Individual components within a chain (like LLMs, tools, or data sources) can be easily swapped or replaced, enabling you to experiment with different configurations and find the best solution.

Common predefined chains are:

- *LLMchain*: It is a basic chain type which takes an input, formats it using a PromptTemplate, and passes it to an LLM. Later, returns the LLM's output.
- *RetrievalQA:* this chain first does a retrieval step to fetch relevant documents, then passes those documents into an LLM to generate a response.
- *QAGenerationChain*: This chain creates both questions and answers from documents.
- *StuffDocumentsChain*: This chain takes a list of documents and formats them all into a prompt, then passes that prompt

to an LLM. It passes ALL documents, so you should make sure it fits within the context window of the LLM you are using.

This framework offers some special functions to load and use predefined chains. For example:

- *create_extraction_chain:* Uses OpenAI Function calling to extract information from text.
- *get_openapi_chain*: Uses OpenAI function calling to query an OpenAPI.
- *create_qa_with_structure_chain:* Uses OpenAI function calling to do question answering over text and respond in a specific format.
- *create_qa_with_sources_chain*: Uses OpenAI function calling to answer questions with citations.
- *load_qa_with_sources_chain:* it does questions answering over documents you pass in and cites their sources. Use this when you want the answer response to have sources in the text response. Use this over RetrievalQAWithSources when you want to pass in the documents directly (rather than rely on a retriever to get them).
- *load_summarize_chain:* Uses a LLM to summarize the text passed in the prompt.

In this lab session we use the *load_summarize_chain to summarize many documents.*

The main parameters used in the *load_summarize_chain are:*

- *llm*: This parameter specifies the Large Language Model to be used in the chain. It is typically an instance of a language model class, such as GPT-3.5 or GPT-4.
- *chain_type*: This parameter determines the type of document combining the chain to use. The "*map_reduce"* approach first summarizes individual documents, then combines these summaries. The "*stuff*" is the simplest approach, where all documents are combined into a single prompt and sent to the LLM in one call. The "*refine*" approach iteratively builds the summary by updating it with each document

In case that the approach used be map_reduce, it is needed to define the prompt used during the map (summarize a document) and the prompt used during the reduce (summarize the summary of all documents):

- *map_prompt*: This is a custom prompt template used for the "map" step of the map_reduce process. It is applied to each document.
- *combine_prompt*: This prompt template is used for the "reduce" step, where the individual summaries are combined into a final summary.

It is possible to use the same prompt during both phases.

Example 11: Using *load_summarize_chain to summarize documents.*

```
# Summarization chain
summarize_chain = load_summarize_chain(llm=self.llm,
chain_type='map_reduce',
map_prompt=selected_prompt,
combine_prompt=selected_prompt
)
# Generate summary
summary = summarize_chain(documents)
print(summary['output_text'])
```

The full list of all predefined chain can be found in the link https://python.langchain.com/v0.1/docs/modules/chains/

ACTIVITY

In the rapidly evolving landscape of natural language processing, efficiently retrieving and synthesizing information from extensive document collections has become increasingly crucial. This assignment aims to develop a simple Retrieval-Augmented Generation system that demonstrates advanced techniques in information extraction and intelligent summarization.

Students have to design and implement a Python-based system, using the API provided by **LangChain**, that can process a collection of news in a .CSV format through a retrieval and summarization pipeline. Students must implement a system capable of accepting natural language queries and dynamically

retrieving the most relevant documents using sophisticated search strategies such as **Semantic Similarity** (SS) and **Maximal Marginal Relevance** (MMR).

The system's summarization capability will be a distinguishing feature, requiring students to generate concise summaries for two distinct audience profiles. The first summary type (**simplified**) targets younger readers, demanding a simplified linguistic approach that transforms complex information into accessible, straightforward language. The advanced type (**advanced**) is tailored for political experts, requiring a more nuanced, sophisticated narrative that captures intricate contextual details.

The implementation demands attention to several components. The document retrieval mechanism must leverage sentence embedding models to convert textual content into dense vector representations, enabling precise semantic matching. The news content must be transformed into a vector using the ***SentenceTransformer***, particularly the **all-MiniLM-L6-v2** model.

Students are required to utilize TokenTextSplitter to fragment the text within news. This step is crucial to ensure that the resulting text segments remain within the context window limitations of the embedding models.

Summary generation utilizes large language model **GPT-3.5-turbo** with the temperature parameter to a low value of **0.3**; the system will minimize model-generated hallucinations, instead prioritizing

direct extraction and intelligent recombination of original content. The summaries are limited to **four** sentences.

The command-line Python program has to provide flexible parameterization, allowing users to specify query texts **(--query/ -q)**, select summarization type **(--stype/-t),** and configure retrieval strategies **(--sretrieval/ -r)**. Students must implement support for semantic similarity searches **(--sretrieval=ss),** where a specified number of top documents can be retrieved **(--ndocuments/ -n),** and maximal marginal relevance **(--sretrieval=mmr)** technique that ensures diversity in the selected documents. In the case of the MMR strategy, value can be specified as --documents that are the retrieved documents, and **(--kmarginal/-k)** that are the number of marginal documents to choose from the n documents retrieved.

python3 rag_summarizer.py **--query** "murder in shooting of black people" **--stype** simplified --**sretrieval** ss **--ndocuments** 10

Simplified Summary: *In the US, police have killed many people, especially African American men who are arrested more often than white men. This has led to the Black Lives Matter movement, which aims to stop police violence and racism. In Chicago, there were 24 shootings and five murders over the weekend, possibly due to gang conflicts. In Texas, a white police officer shot and killed a black woman in her home, leading to his resignation and murder charges.*

python3 rag_summarizer.py **--query** "murder in shooting of black people" **--stype** advanced **--sretrieval** mmr **--ndocuments** 10 --**kmarginal** 5

Advanced Summary: *The data on police killings in the US reveals a troubling pattern of violence and systemic injustice, particularly towards marginalized communities like African Americans. This highlights the deep-rooted racial biases within the criminal justice system and the urgent need for accountability and reform. The emergence of movements like Black Lives Matter signifies a critical response to these injustices, calling for a*

comprehensive examination of power dynamics and structural inequalities in society.

This multi-dimensional analysis illuminates the complex interplay of historical, social, and political factors that perpetuate the cycle of trauma and injustice experienced by communities of color, emphasizing the need for holistic solutions to address the root causes of violence and inequality.

Deliver the source code and prepare a comprehensive report explaining your approach, and findings. The Google collaborative environment (Google Colab) can be used to implement the RAG system:
https://colab.research.google.com/notebooks/welcome.ipynb#scrollTo=5fCEDCU_qrC0